

k-playbook

Guardrails & Workflow für AI-gestützte Entwicklung

Drei Säulen:

1. **Enforcement** bestehender Vorgaben (Styleguides, Doku, Konventionen)
2. **Kontext bereitstellen** – interne + externe Doku, einfacher Zugriff (optional: RAG / MCP)
3. **Umsetzung** – geplante Änderungen strukturiert & sicher erzeugen (Task-Pipeline, Git-Diff-Protokoll, Code-Review, Checks, Deploy)

Ziel: Ein wiederverwendbares Set aus Skills, Commands & Templates, das in beliebigen Repos „installiert“ werden kann (siehe [install.md](#)).

Einordnung

Ich stelle in dieser Session **mein k-playbook** vor — ein Set aus Regeln, Commands und Skills, das ich über die **letzten Jahre** aus konkreten Projekten heraus entwickelt und iterativ verfeinert habe.

- **Allgemein genug**, um in ganz unterschiedlichen Projekten zu funktionieren.
- **Leicht anpassbar** — projektspezifische Teile (Guidelines, Checks, Reviews) liegen im jeweiligen Repo, das Playbook selbst bleibt schlank.
- Ursprünglich für **Claude Code** geschrieben, funktioniert aber genauso gut mit **OpenCode**. (Portabilität ist bewusst mitgedacht — [AGENTS.md](#), Slash-Commands und Skill-Struktur sind kompatibel.)

Kein akademisches Framework — sondern gewachsene Praxis, die aus dem Alltag stammt und sich bewährt hat.

Motivation

Bei **großen Projekten** — übernommenen wie eigenen — entstehen drei Kernprobleme parallel:

A · Mensch verliert den Überblick

- Struktur, Konventionen und implizites Wissen sind nur teilweise dokumentiert.
- Regeln existieren oft nur „in Köpfen“ – Neuzugänge (Mensch *oder* KI) brechen sie unwissentlich.
- Es fehlt eine Basis, um später **verbindliche Regeln zu definieren** und deren Einhaltung zu erzwingen.

B · KI arbeitet ineffizient und unzuverlässig

- Ohne Projekt-Kontext liest die KI in jeder Session dieselben Dateien neu → **Token-Verschwendung**.
- Fehlende Leitplanken führen zu **Halluzinationen** und dazu, dass sich die KI in Sackgassen **verrennt**.
- Bibliotheks-Wissen ist veraltet → Anti-Patterns, deprecated APIs, Sicherheitslücken.

C · Änderungen laufen unstrukturiert und unnachvollziehbar

- Ad-hoc-Chats produzieren Code ohne klar dokumentiertes **Ziel** und ohne saubere **Change-History**.

- **Scope-Creep** und beiläufige „Mit-Änderungen“ werden im Nachhinein nicht mehr entwirrt.
- Zwischen „KI hat committed“ und „geht live“ fehlt ein **verlässliches Prüf-Netz**.

Ziel des k-playbook: Ein installierbares Set aus Skills, Commands und Templates, das

1. **Regeln definierbar und durchsetzbar** macht (Enforcement),
2. **Kontext bereitstellt** — für Mensch (Onboarding, Überblick) und KI (weniger Tokens, weniger Halluzinationen),
3. **Änderungen strukturiert und nachvollziehbar** erzeugt — vom Task über den Diff bis zum Deploy.

Überblick – Die drei Säulen

Säule	Frage	Ergebnis
1 • Enforcement	Wie erzwingen wir bestehende Vorgaben?	Global + projekt-lokale Regeln, k-enforcement -Skill, Checks
2 • Kontext bereitstellen	Wie greifen Mensch & KI einfach auf projektinternes und externes Wissen zu?	Kuratierte Doku + Pitfall-Kataloge; optional RAG / MCP-Server
3 • Umsetzung	Wie erzeugen wir Änderungen strukturiert & sicher?	Task-Pipeline (create → review → run → code-review), Git-Diff-Protokoll, Deploy-Gate

Säule 1 – Enforcement bestehender Vorgaben

Fokus: KI-gestütztes Enforcement

Zusätzlich zu den klassischen Enforcement-Ebenen (Editor/LSP/Formatter, Pre-commit-Hooks & Commit-Konventionen, CI/CD mit Quality-Gates & Policy-as-Code) **beschäftigen wir uns hier primär mit den Ebenen, bei denen KI und Enforcement sich gegenseitig unterstützen** — also Regeln, die die KI aktiv befolgen muss, und Prüfungen, die durch KI erst effizient möglich werden.

Ablage der Regeln – global + projektspezifisch

Wir nutzen **nicht AGENTS.md** als primäre Regelquelle, sondern eine zweistufige Ablage:

- **Global** (im k-playbook: **enforcement/**) Regeln, die für **alle** Projekte gelten: Dokumentations-Konventionen, allgemeine Do/Don'ts, generelle Arbeitsweise.
- **Projekt-lokal** (Verzeichnis im Repo, z.B. **guidelines/** oder **styleguides/**) Projekt-spezifische Styleguides, Naming-Conventions, Architektur-Entscheidungen.

AGENTS.md verweist nur auf diese beiden Quellen (Pointer statt Inhalt) — so bleibt es kurz und wartbar, und die Regeln sind an einem einzigen Ort.

Umgang mit Legacy-Code: Beide Ablageorte müssen berücksichtigen, dass Bestandscode die neuen Regeln oft (noch) verletzt. Praktisch heißt das: **Baseline-Files** und **Ratcheting** (à la `eslint --report-unused-disable-directives`) — Regeln gelten für Änderungen, nicht rückwirkend für den gesamten Bestand.

Der Skill **k-enforcement**

Zentraler Baustein: Der Skill **k-enforcement**

- Prüft **laufend während der Erstellung**, dass die Regeln aus global + projekt-lokal tatsächlich eingehalten werden.
- Kein einmaliger Kontext-Push zu Beginn, sondern **kontinuierliche Prüfung** im Arbeitsprozess.
- Ergänzt (nicht ersetzt) die späteren Kontrollen durch `/k-review-loop`, `/k-code-review` und die `checks/`.

Kern-Trennung:

- Skills → Regel-Einhaltung **während** der Erstellung
 - Checks → Verifikation **danach** (weil Sollen ≠ Tun, siehe Checks-Abschnitt)
-

Säule 2 – Kontext bereitstellen

Daten vorhalten & Zugriff so einfach wie möglich gestalten

Prinzip

AI-Sessions starten **ohne Gedächtnis**. Ohne kuratierten Kontext liest die KI in jeder Session dieselben Dateien neu — oder rät, wenn die Suche zu aufwändig wird.

Zwei Datenquellen zusammen ergeben den Kontext:

- **Intern:** Projekt-Doku, ADRs, Konventionen, README
- **Extern:** Libraries & APIs mit ihren Stolperfallen (Version-Pinning, Breaking Changes, Auth-Quirks, ...)

Kern: Daten **einmal kuratieren**, dann so ablegen und indexieren, dass Mensch *und* KI **so einfach wie möglich** darauf zugreifen können.

Interne Kontext-Basis

- **AGENTS.md** im Repo-Root — Kontext-Anker, wird automatisch geladen (Claude Code / OpenCode)
- **Doc-Index** (`docs/README.md`) mit **Keyword-Tags** pro Datei — schnelles Auffinden
- **Doku-Aufbereitung** als Doc-as-Code:
 - Chunking langer Dokumente (header-basiert / semantic)
 - Frontmatter-Metadaten (`tags`, `scope`, `last-reviewed`)
 - Cross-Linking über relative Pfade
 - **ADRs** (Architecture Decision Records) im `docs/adr/`-Format

Prinzip: Doku wird einmal kuratiert → dauerhaft für alle Sessions verfügbar.

Referenz-Playbook: [ks-ai-session-memory/](#)

Externe Kontext-Basis (Libraries & APIs)

Problem: LLMs kennen Libraries nur bis zum Training-Cutoff. Breaking Changes, deprecated APIs und Stolperfallen (Rate-Limits, Timezone-Bugs, Non-Idempotenz) stehen selten prominent in der offiziellen Doku.

Kuratierte Pitfall-Kataloge pro Library:

- Ablage in [docs/libs/<name>.md](#) mit Frontmatter ([lib](#), [version](#), [severity](#), [last-reviewed](#))
- Inhalt: Version-Range, Migration-Notes, **Pitfall-Katalog** (Auth, Concurrency, Perf, Security), empfohlene Idioms + Anti-Patterns
- Quellen: offizielle Docs, Changelog, GitHub-Issues ([bug](#), [gotcha](#)), Stack Overflow, OSV/GHSA für Security

Erzeugung: Command [/k-tools-scan](#) — erkennt aus Manifest-Files und Code-Nutzung die genutzten Tools, scored sie („empfohlen / optional / skip“), der User bestätigt die Auswahl, dann pro Tool eine Datei mit gezielter (nicht breiter) Web-Recherche. Läuft üblicherweise nach [/k-code2docs](#).

Fokus: Pitfalls, nicht Copy-Paste-Snippets.

Optional: Zugriffs-Methoden bei größeren Datenmengen

Wenn die kuratierten Docs zu umfangreich für das direkte Kontext-Fenster werden, gibt es **mehrere Optionen** — kein Zwang zu einem bestimmten Weg:

- **RAG (Retrieval-Augmented Generation)** — Embeddings + Vector Store ([sqlite-vec](#), Qdrant, Chroma, LanceDB); Hybrid-Retrieval (BM25 + Dense) für Fachbegriffe; alles offline betreibbar
- **MCP-Server** — z.B. **Context7** für Live-Library-Docs, oder projekteigene MCP-Server, die kuratierte Inhalte on-demand liefern
- **Suchbasierte Tools** — ripgrep über [docs/](#), DevDocs.io, [docs.rs](#), [pkg.go.dev](#) — reicht oft völlig
- **Kombinationen** — z.B. Doc-Index (Sparse) + gezielter File-Read

Faustregel: Erst prüfen, ob klassisches File-Read reicht — sonst das **einfachste** Werkzeug wählen, das trägt. Nicht der volle RAG-Stack, wenn ein MCP-Server oder ripgrep den Job macht.

Säule 3 – Umsetzung

Änderungen strukturiert & nachvollziehbar erzeugen

Die ersten beiden Säulen sorgen dafür, dass **Regeln und Kontext** verfügbar sind. Säule 3 nutzt das, um **konkrete Änderungen am Code kontrolliert durchzuführen** — von der Idee bis zum Deploy-Gate.

Bausteine (im Folgenden je ein eigener Abschnitt):

Baustein	Zweck
Tasks (/k-create-task)	Aus Gespräch → kuratierte, self-contained Task-Datei
Reviews (/k-review-loop)	Perspektiv-Umkehr auf den Task, bevor Code entsteht
Ausführung (/k-run)	Sequentielle Umsetzung, Git-protokolliert
Code-Review	Findings auf dem Diff (nach der Ausführung)
Checks	Projektspezifische Verifikation (Sollen ≠ Tun)
Pre-Deploy-Check	Hartes Gate vor dem Deploy — nur Show-Stopper

Roter Faden: Alle Bausteine arbeiten auf **derselben Task-Datei** — Intent + Referenzen werden einmal kuratiert und ziehen sich durch die ganze Kette.

Tasks

Warum ein eigenes Task-Format?

Übliche Praxis: Aufgabe direkt im Chat besprechen → Agent los. **Problem:** Der ausführende Agent muss

- **selbst suchen**, welche Dateien / Docs relevant sind → Tokens verbrannt, Kontext schwillt an.
- **raten**, wenn die Suche zu aufwändig wird → Halluzinationen, Anti-Patterns.
- **improvisieren**, was das eigentliche Ziel („Intent“) ist → Sackgassen, Rework.

Lösung im k-playbook: Der Task ist ein **kuratiertes, self-contained Dokument** — erzeugt aus dem Gespräch, bevor der ausführende Agent überhaupt startet.

/k-task-create – was der Command tut

Aus der laufenden Konversation wird eine strukturierte Task-Datei erzeugt:

- **Nummerierung** automatisch (tasks/ + tasks/old/ gescannt → nächste freie Nummer, 014-audiosocket-server.md)
- **## Intent** — 1–2 Sätze Zielrahmen + 2–5 Erfolgskriterien (bei Task-Serien nur im **letzten** File — dient als Anker für Alignment)
- **## Referenzen** — alle im Gespräch erwähnten relevanten Dateien/Docs mit Begründung
- **## Tools** — nur **zusätzliche** Tools über Standard-Set hinaus (MCP, spezielle Permissions)
- **## Ziel / ## Kontext / ## Zu bauen** — was, warum, konkret welche Artefakte
- **Confirm-Loop** — Draft wird gezeigt, User bestätigt, dann erst gespeichert

Ergebnis: Der Task enthält bereits alles, was der ausführende Agent (und der Reviewer) braucht.

Wirkung: weniger suchen, nicht raten

Token-Effizienz

- Referenzen sind kuratiert → Agent liest gezielt, nicht explorativ.
- Kontext wird **einmal** vom Menschen zusammengestellt statt **jedes Mal** von der KI erneut erschlossen.

Weniger Halluzinationen

- Der klassische Auslöser: „Suche wäre zu teuer → ich rate mal plausibel.“
- Wenn die Quelle direkt im Task steht, entfällt der Anreiz zur Improvisation.

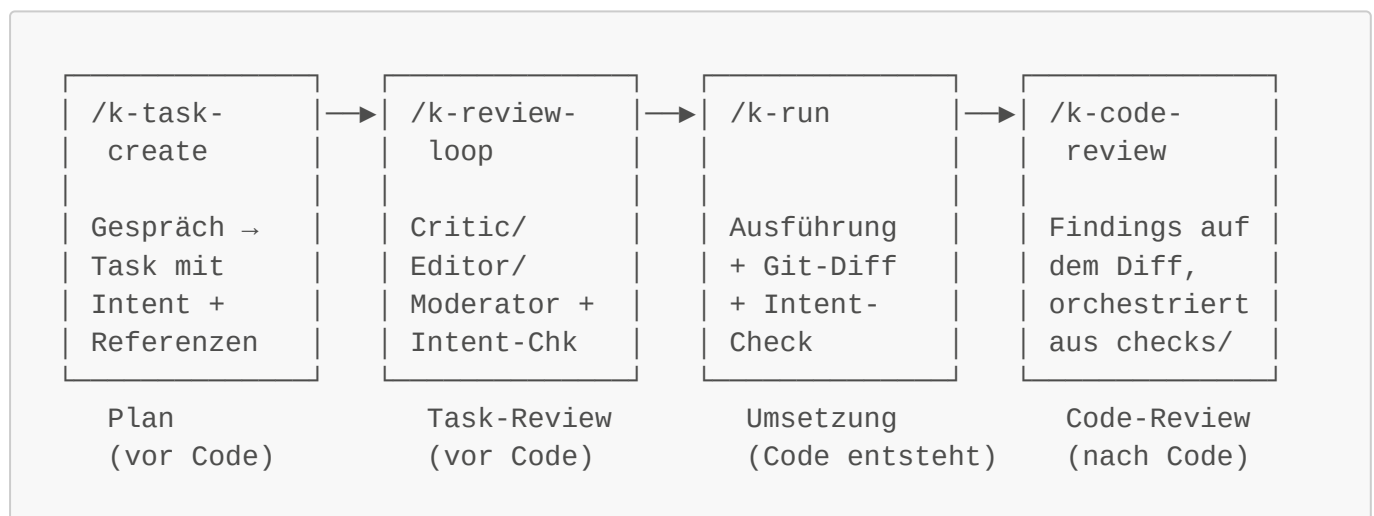
Weniger Verrennen

- **Intent** gibt harten Erfolgsmaßstab → Agent kann Sackgassen frühzeitig erkennen.
- **Zu bauen** grenzt den Scope ab → keine Feature-Creep-Rabbit-Holes.

Doku-Nachzug als Teil des Tasks

- Zu jedem Task gehört: am Ende die **Doku nachziehen**.
- Bei größeren Änderungen **mit Rückfrage** an den User (was gehört ins ADR, was in die Referenz-Doku?).
- Verhindert, dass Doku und Code auseinanderlaufen — die spätere Prüfung (siehe *Checks*) hat dann überhaupt eine Chance.

Die Pipeline: create → review → run → code-review



Zwei Reviews mit unterschiedlichem Gegenstand — bewusst unterschiedlich benannt:

- **Review** (`/k-review-loop`) prüft den **Task** — bevor Code entsteht. Frage: „Erreicht der Plan sein Intent?“
- **Code-Review** (`/k-code-review`) prüft den **Diff** — nachdem Code entstanden ist. Frage: „Sind in dem, was tatsächlich geschrieben wurde, Probleme?“

Entscheidend: Alle vier Schritte arbeiten auf **derselben** Task-Datei. **## Intent** und **## Referenzen** aus Step 1 sind die Anker für alle nachfolgenden Schritte — kein doppeltes Zusammensuchen, kein Format-Bruch zwischen den Phasen.

Reviews

Der übliche Ansatz (und warum er teuer ist)

Viele AI-Review-Setups setzen auf:

- **Mehrere LLMs parallel** (Claude + GPT + Gemini ...) → Kosten × N, Latenz × N
- **Verschiedene Personas** („Security-Reviewer“, „Senior-Dev“, „QA“) auf demselben Prompt
- **Consensus-Voting** über die Ergebnisse

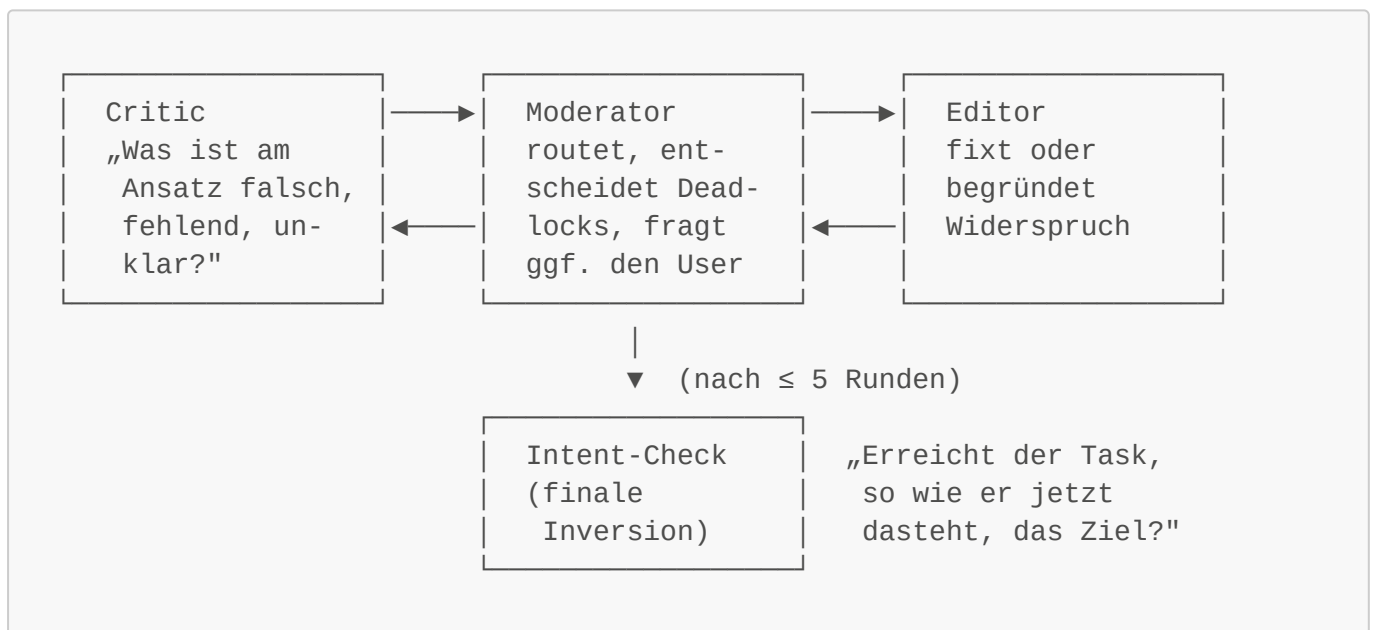
Problem: Die eigentliche Motivation ist ein **Perspektivwechsel** auf den Code — aber Personas und Modellwechsel liefern das nur unzureichend. Die Modelle sehen dieselbe Aufgabe aus demselben Blickwinkel — nur mit anderem „Kostüm“.

Der k-playbook-Ansatz: Perspektivwechsel durch Aufgabenumkehr

Grundidee: Nicht das *Modell* wechseln, sondern die **Aufgabenstellung drehen**. Dadurch wird der Blickwinkel **strukturell** anders — auch mit demselben (günstigen) LLM.

Konkret umgesetzt in **/k-review-loop**

Review von **Task-/Instruction-Dateien vor der Ausführung** — strukturierter Dialog:



Kern-Inversion: Nicht „schreibe den Task“, sondern „**prüfe, ob dieser Task sein eigenes Intent erreichen kann**“ — ein struktureller Rollentausch.

Design-Details von **/k-review-loop**

- **Drei Rollen, ein Modell:** Critic · Editor · Moderator — als separate Subagent-Turns, jede mit eigenem Prompt/Blickwinkel.
- **High-Level-Review:** Prüft **Ansatz, Widersprüche, fehlende Constraints** — nicht Style oder Details. Der ausführende Agent füllt Details.
- **Kategorien:** **FEHLER** (blockierend) · **WARNUNG** · **FEHLEND** — Moderator filtert, nur **FEHLER** gehen automatisch in die Editor-Runde.

- **Deadlock-Handling:** Bis zu 5 Runden Critic ↔ Editor; wiederholt sich das Argument → Moderator entscheidet und dokumentiert.
- **Intent als externer Anker:** Inline im Task oder als Datei-Referenz ([requirements.md](...)). Der finale Alignment-Check fragt nur: „erreicht der Task sein Intent, ja/nein?“
- **Nachvollziehbarkeit:** Discussion-Log wird an die Task-Datei angehängt (Diskussion, Moderator-Entscheidungen, Alignment-Ergebnis, offene Punkte).
- **Optionale Lenses (fachliche Perspektiven):** Per Frontmatter review-lens: [architect, security, ...] im Task aktivierbar. Lenses laufen parallel zum generischen Critic, jede mit einem eigenen Prompt-Template (z. B. review/lenses/architect.md). Moderator dedupliziert Findings vor der Editor-Runde. Default (kein Frontmatter) = generischer Critic wie bisher.

Weitere Perspektiv-Umkehrungen (Muster)

Dasselbe Prinzip lässt sich auf weitere Phasen anwenden:

Rolle A (produzierend)	Rolle B (invertierend)
„Schreibe diesen Task“	„ Erreicht der Task sein Intent?“ (← /k-review-loop)
„Implementiere Feature X“	„Welche Tests würden diese Implementierung <i>widerlegen</i> ?“
„Schreibe diese Funktion“	„Wie würdest du sie missbrauchen / zum Absturz bringen?“
„Refactor dieses Moduls“	„Welche Regressionen kann dieser Refactor auslösen?“
„Schreibe die Doku dazu“	„Verstehe nur aus der Doku, was der Code tut — was fehlt?“
„Erkläre den Bug-Fix“	„Reproduziere den Bug aus dem Fix heraus — passt die Kausalität?“

Die zweite Achse ist **fachlich** — dieselbe Phase, aber unter einer bestimmten Lens:

Lens	Kernfrage	Typischer Task
architect	Passt das zu Systemgrenzen und Kopplung?	Refactor, Cross-Module
security	Wo sind Auth/Injection/Secret-Leaks?	I/O, Auth-Flows
qa	Womit widerlege ich das? Welche Edge-Cases?	Bug-Fix, Features
ops	Wie deploye/rolle ich zurück? Prod-Impact?	Migration, Infra
performance	Skaliert das? N+1? Memory?	Data-Heavy
ux	Ist die CLI/API/Fehlermeldung verständlich?	User-facing

Lenses sind **kombinierbar** mit Umkehrungen: Ein Refactor-Task kann sowohl architect (Fach-Lens) als auch die Regressions-Umkehrung durchlaufen.

Kern: Der Rollentausch ist im **Task** kodiert, nicht in einer Persona-Beschreibung. Fachliche Lenses (Tabelle oben) sind **kein Widerspruch dazu**: Sie sind ebenfalls fest formulierte Prompt-Templates, keine „Charaktere“ mit Beziehungen. Der Unterschied liegt nur im fachlichen Fokus (z. B. Security-Fragen vs. generische Kritik).

Warum das billiger *und* effizienter ist

- **Ein Modell reicht** — auch ein kleineres/günstigeres.
- **Kein Consensus-Overhead** — zwei komplementäre Turns statt N paralleler.
- **Strukturelle Diversität** statt oberflächlicher (Kostüm-)Diversität.
- **Reproduzierbar**: Die Umkehrung ist im Prompt-Template festgeschrieben, nicht im „Bauchgefühl“ eines Personas.
- **Composable**: Mehrere Umkehrungen lassen sich verketteten (Vorschlag → Prüfung → Gegenprüfung).

Umsetzung im k-playbook

- **/k-review-loop <path>** (vorhanden)
 - Pre-Execution-Review von Task-/Instruction-Dateien
 - Critic ↔ Editor ↔ Moderator, bis zu 5 Runden
 - **Intent-Alignment** als finaler Check (Inline-Text oder Datei-Referenz)
 - Discussion-Log wird in die Task-Datei geschrieben (Audit-Trail)
 - Optionale **Lenses** aus **review/lenses/<name>.md** (fachliche Prompt-Templates, parallel zum Critic)
- **Skill ks-review-flip** (geplant)
 - deckt die **Phasen-Achse** ab (Implementation, Refactor, Doku, Bug-Fix); die **Lens-Achse** (fachliche Perspektive) ist orthogonal dazu und im Review-Loop verankert — beide kombinierbar
 - überträgt das Muster auf **weitere Phasen** (Implementation, Refactor, Doku, Bug-Fix)
 - liefert Prompt-Templates für die häufigsten Rollen-Umkehrungen
 - Verkettung mehrerer Umkehrungen (Producer → Inverter → Judge)
- **Integration mit Säule 1**: Wiederkehrende Findings aus Review-Logs → neue Do/Don't-Einträge in **AGENTS.md** (Regeln wachsen aus Beobachtung).

Ausführung (/k-run)

Was /k-run tut

Führt eine oder mehrere Task-Dateien sequentiell aus — mit strikter Absicherung drumherum:

- **Reihenfolge nach Nummer** (013-... vor 014-...), **nie parallel** (zwei Agents dürfen nicht gleichzeitig Code ändern)
- **Tools-Upfront-Anzeige** — alle zusätzlichen Tools/Permissions aus allen Tasks werden **vor Start** gesammelt und dem User angezeigt (keine Nachfragen mitten im Lauf)
- **Klärung im Main-Context** (Step 2b) — offene Fragen werden **vor** der Delegation an den Sub-Agent gestellt; der Sub-Agent darf explizit nicht raten
- **Sub-Agent-Isolation** — jede Task-Ausführung in eigenem Sub-Agent, Blocker werden eskaliert statt „quick-and-dirty“ gelöst
- **Erfolg** → **move nach done/**, Fehler → Datei bleibt liegen mit Abbruch-Notiz
- **Finaler Intent-Alignment-Check** nach allen Tasks: derselbe Check wie in **/k-review-loop**, jetzt gegen die **tatsächliche Ausführung** statt gegen den Plan

Git-Absicherung & Protokoll (Kern-Vorteile)

Der Command nutzt Git als **Wahrheits-Anker** — jeder Task hinterlässt einen nachvollziehbaren Fußabdruck:

Ablauf pro Task:

1. Vor Start: `BASELINE_HASH = git rev-parse HEAD` festhalten
2. Optional: `before:-`Kommando aus `CLAUDE.md` (z.B. `make sichern` als Backup)
3. Task wird ausgeführt
4. Nach Erfolg: `git diff --stat` + `git diff $BASELINE_HASH` erzeugen
5. Diff wird **direkt in die Task-Datei** unter `## Ausführung` geschrieben
6. Optional: `after:-`Kommando (z.B. erneutes Backup / Deploy-Trigger)

Warum das entscheidend ist:

- **Nachweis „nur was zu tun war, wurde getan“** — der Diff ist per Konstruktion die vollständige Menge aller Änderungen dieses Tasks. Scope-Creep wird sichtbar.
- **Audit-Trail an der Quelle** — die geänderten Dateien stehen in derselben Datei wie Task, Intent, Ausführungs-Status. Kein externes Logging-System nötig.
- **Reproduzierbarkeit** — Baseline-Hash + Diff + `before:/after:-`Hooks = exakt rekonstruierbar, was passiert ist.
- **Rollback-fähig** — bei unerwünschten Änderungen genügt `git reset $BASELINE_HASH`.
- **Automatisierte Backups** über den `before:-`Hook (z.B. `make sichern` vor jeder Task-Ausführung) — kein manueller Schritt vergessbar.
- **Selbstbeschränkung des Reviews** — der spätere Code-Review bekommt **nur** den Diff, keine explorative Codebase-Erkundung. Weniger Tokens, klarer Scope.

Code-Review – Ansatz

Prinzip: Review-Regeln sind **projekt- und sprachspezifisch** → keine allgemeingültige Lösung sinnvoll. Das k-playbook liefert die **Orchestrierung**, die Review-Dateien liefern den **Inhalt**.

- **Ablage** wahlweise **projekt-lokal** (`reviews/`, konfiguriert in `K-PLAYBOOK.MD`) oder **global** im Playbook-Repo. Bei Namenscollision gewinnt projekt-lokal.
- **Review-Dateien sind pfad-frei**. Sie beschreiben nur *was* geprüft wird, Stil-Wahl und Beispiele. Keine Verweise auf Log-Datei, `known-decisions.md` oder Ergebnis-Pfade.
- **Auflösung durch den Command** (`/k-review`): liest `K-PLAYBOOK.MD`, ermittelt `<reviews>/log.md`, `<reviews>/known-decisions.md`, ggf. `<reviews>/result-<name>.md`. Wird ein Pfad einmal umbenannt, betrifft es nur den Command — nicht N Review-Dateien.
- **Frontmatter** pro Review liefert Metadaten (`title`, `interval-weeks`, `scope-hint`, optional `handoff`) — deterministisch statt Prosa.

Auswertung & Umsetzung

Zwei Bausteine reichen für diese Präsentation aus:

1 • Auswertungs-Command (`/k-review`)

- Startet eine Review-Session gegen ein Review-Rezept (projekt-lokal oder global — Auswahlliste, wenn kein Name übergeben wird).
- Löst Pfade aus **K-PLAYBOOK.MD** auf (`<reviews>/log.md`, `known-decisions.md`).
- Geht **jeden Findpunkt der Reihe nach** durch — moderiert, keine Batch-Ausgabe.
- Aktualisiert am Ende die Log-Datei (Datum, Fällig-ab, Protokoll-Zeile).

2 • Anschluss nach der Besprechung

Nach dem Durchgang werden die zu adressierenden Punkte auf zwei Wege verteilt:

- **Einfache Sachen** → Task-Sammlung erzeugen und über den **üblichen Workflow** lösen (`/k-task-create` → `/k-review-loop` → `/k-run`).
- **Komplexere Themen** → **spezielles Bewertungs-/Besprechungs-Verfahren** (*Details folgen in einem eigenen Kapitel.*)

Kern: Der Review-Prozess mündet strukturiert zurück in die bestehende Pipeline — Findings werden nicht „irgendwie fixieren“, sondern als saubere Tasks mit Intent und Referenzen durchgereicht.

Checks

Was ist ein „Check“?

Neben den generischen Playbook-Bausteinen hat jedes Projekt ein **projekt-spezifisches Verzeichnis checks/** — eine Sammlung aus **.md-Dateien, Shell-Skripten und Kommandos**, die je nach Anforderung aufgerufen werden.

Spannweite:

- **Einfach & schnell:** Unit-Tests starten, Smoke-Tests, Lint-Runs
- **Aufwändiger:** Prüfungen, dass vorher festgelegte **Erfordernisse tatsächlich eingehalten** werden
 - z.B. Doku ↔ Code-Konsistenz (in **beide** Richtungen)
 - Doku widerspricht sich nicht selbst
 - Konventionen (Naming, Struktur) werden im Bestand eingehalten
 - projektspezifische Invarianten (Migrations vollständig, keine Dead Code Pfade, ...)

Wichtige Abgrenzung zu Skills: Skills sollen bei der **Erstellung** helfen, dass Vorgaben eingehalten werden. Das heißt aber noch lange nicht, dass das tatsächlich passiert. **Checks verifizieren im Nachhinein** — sie sind das objektive Kontrollnetz.

Ausführungs-Modi

Checks unterscheiden sich stark in Laufzeit — manche sind Sekunden, manche Minuten oder länger. Deshalb wird beim Aufruf **immer der Modus abgefragt:**

- **schnell** — nur die günstigen Checks (z.B. Unit-Tests, statische Prüfungen)
- **umfassend** — komplettes Set inkl. langlaufender Prüfungen (Doku-Konsistenz, End-to-End)
- **Auswahl** — gezielte Teilmenge (z.B. nur die Checks, die zum aktuellen Change passen)

Zusätzlich lässt sich beim Aufruf **explizit angeben**, welche Checks laufen sollen — für gezielte Deep-Checks eines bestimmten Themas.

Ergebnis-Aggregation: Die Ergebnisse aller gelaufenen Checks werden strukturiert vorgestellt (analog zum Review-Auswertungs-Command — jeder Befund einzeln, der Reihe nach).

Pre-Deploy-Check

Absicherung vor dem Deploy

Bevor Code nach Dev / Prod geht, wird ein **orchestrierter Check** ausgelöst — **Checks + Reviews gemeinsam**, aber mit **anderer Filterung** als im normalen Alltag.

Ablauf:

1. **Checks komplett aufrufen** (umfassender Modus)
2. **Reviews starten**
3. Ergebnisse strikt gefiltert präsentieren (siehe nächste Slide)

Ziel: nichts wird veröffentlicht, was nachweislich blockiert — aber der Prozess erstickt nicht in Verbesserungsvorschlägen.

Der Deploy selbst ist **nicht Teil** des k-playbook — er läuft weiterhin über die projekt-eigenen Deploy-Mechanismen (CI/CD, Skripte, Pipelines). Das Playbook liefert nur das **Gate davor**.

Pre-Deploy-Filter: nur Show-Stopper

Beide Quellen — Checks und Reviews — liefern normalerweise **viele Findings**. Vor einem Deploy interessieren aber **nur die harten**:

Reviews im Pre-Deploy-Modus:

- □ Tatsächlich **entscheidende Fehler** werden aufgelistet
- × Verbesserungsvorschläge, Style, „Nice-to-Have“ **fallen weg**

Checks im Pre-Deploy-Modus:

- □ Alles, was gegen eine Veröffentlichung **spricht** oder sie **verhindert**
- × **Keine Verbesserungen** — nur Blocker

Warum diese Härte-Filterung?

- Wenn das Gate mit Style-Diskussionen zugemüllt ist, wird es ignoriert.
 - Der Filter macht das Gate **glaubwürdig**: was durchkommt, ist wirklich deploy-fähig.
 - Verbesserungen gehen ihren normalen Weg über Task-Sammlungen (siehe Reviews-Abschnitt) — nicht am Gate blockierend.
-

Vor größeren Deploys (oder zwischendurch)

Zusätzlich zum blockierenden Gate: **umfassender Check** vor größeren Meilensteinen — mit Fokus auf **Konsistenz**:

- **Doku intern** — widerspricht sich die Doku selbst?
- **Code ↔ Doku (beide Richtungen)**
 - Beschreibt die Doku noch, was der Code tut?
 - Wird alles, was der Code tut, von der Doku erfasst?
- Wiederkehrende Invarianten aus **checks/**

Kopplung zurück zu den Tasks: Weil zu jedem Task der **Doku-Nachzug** gehört (siehe Tasks-Abschnitt), hat dieser Check überhaupt eine faire Chance — sonst wäre das Delta zwischen Code und Doku nach kurzer Zeit unaufholbar.

Installation

/k-setup – ein Command, ein Einstieg

Die Installation des k-playbook in einem Projekt läuft über einen einzigen Command:

```
/k-setup
```

Was passiert dabei:

1. **Erkennung des Ziel-Systems** — je nachdem, ob die Session in **Claude Code** oder **OpenCode** läuft, werden die passenden Skills und Commands am richtigen Ort eingerichtet.
2. **Einrichtung im Projekt** wird angeboten — kein Zwang, aber der Standardpfad.
3. **Pfade werden vorgestellt** — alle Verzeichnisse, die das Playbook nutzt (**tasks/**, **checks/**, **reviews/**, **guidelines/**, **enforcement/**, ...).
 - Jeder Pfad kann **umbenannt** oder **an einen anderen Ort** verlegt werden.
 - Bei Abweichungen werden die zugehörigen Beschreibungen und Templates **automatisch angepasst**.

Ergebnis: das Playbook fügt sich in bestehende Projekt-Konventionen ein, statt sie zu überschreiben.

K-PLAYBOOK.MD – die Zentrale im Projekt

Alle Entscheidungen aus **/k-setup** (gewählte Pfade, aktivierte Bausteine, projektspezifische Anpassungen) landen in einer einzigen Datei im Projekt-Root:

K-PLAYBOOK.MD (bewusst großgeschrieben — Signal-Datei)

- **Config-/Pointer-Datei, keine User-Doku.** Enthält keine Erklärungen für Menschen, sondern die Pfade und Aktivierungs-Zustände, aus denen die Commands (**/k-run**, **/k-review**, **/k-task-create**, ...) ihre Konfiguration lesen.

- **Single Source of Truth für Pfade.** Auch wenn ein Pfad dem Default entspricht, steht er drin — spätere Umbenennungen wirken zentral.
- Ein `<!-- k-setup:managed -->`-Block markiert den verwalteten Bereich; alles außerhalb bleibt vom User (Notizen, Team-Hinweise).

Nachträgliche Anpassung: `/k-setup` kann jederzeit **erneut** aufgerufen werden — Pfade ändern, neue Bausteine aktivieren, alte deaktivieren. Der managed-Block wird konsistent nachgezogen, User-Anteile bleiben unberührt.

-
- **Governance:** Wer pflegt die Pitfall-Kataloge? Review-Kadenz?
 - **Verteilung:** Framework als **Template-Repo**, **degit-Snapshot** oder **Git-Submodule**?
 - **Portabilität:** OpenCode-Skills ↔ Claude-Code-Skills ↔ Cursor-Rules ↔ generisches `AGENTS.md`?
 - **Messbarkeit:**
 - Wie messen wir Wirkung? (Anzahl korrigierter Anti-Patterns, weniger Review-Runden, ...)
 - Baseline-Metriken vor/nach Einführung?
 - **Datenschutz:** Welche Doku darf in Cloud-Embeddings? Was bleibt strikt lokal?
-